ADPro for Robotic Diffusion Policies

Algorithm Steps and Code-Level Implementation

SIMUL Project – Franka Panda

March 31, 2026

1 Objective

ADPro (*Adaptive Diffusion Policy*) is a *test-time* adaptation method for a pre-trained diffusion policy. The main idea is to avoid re-training and instead guide denoising using task geometry.

In this project, ADPro is implemented in `models/adpro.py` through the `ADPro` class, with two variants:

- `implementation='paper'`: most faithful version to the paper (FGR + Chamfer guidance + spherical constraint).
- `implementation='practical'`: more robust/pragmatic simulation version (explicit target + optional blend).

2 Notation

- Observation: $O = (O^0, O^1)$ where O^0 is the gripper point cloud and O^1 is the scene point cloud.
- 7D action: $a = [x, y, z, q_x, q_y, q_z, q_w]$.
- Diffusion step: t (from high noise to low noise).
- Base policy: network $\epsilon_\theta(O, a_t, t)$.

3 Step 1 – Task-Aware Initialization

3.1 Theory

Instead of starting from pure Gaussian noise $a_T \sim \mathcal{N}(0, I)$, ADPro computes a structured initial pose a_0^{est} from scene geometry, then projects it to noise level $t = M$:

$$a_M = \sqrt{\bar{\alpha}_M}\, a_0^{\text{est}} + \sqrt{1 - \bar{\alpha}_M}\, \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \tag{1}$$

3.2 Implementation

Function: `_task_aware_init(obs, t_start)`.

- Point subsampling: `subsample_pointcloud`.
- Object-point filtering (to reduce table influence): z-based mask.
- `paper` variant: full rigid estimate with `fast_global_registration`, then `se3_to_action`.
- `practical` variant: explicit grasp target (via `_target_grasp_action`) + centroid-based correction.

4 Step 2 – One Diffusion Denoising Step

4.1 Principle

The standard DDPM reverse step is first applied to obtain $\tilde{a}_{t-1}$.

4.2 Implementation

In `_adpro_step`:

- Noise prediction: `eps_pred = policy.network.forward(obs, a_t, t)`.
- DDPM step: `a_tilde = policy.ddpm_step(...)`.
- Deterministic option: `deterministic_denoising=True` removes reverse-process stochastic noise for stability.

5 Step 3 – Clean Action Estimation (Tweedie-like Formula)

$$\hat{a}_0 = \frac{a_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta(O, a_t, t)}{\sqrt{\bar{\alpha}_t}}. \tag{2}$$

Implementation: `_predict_clean_action`.

6 Step 4 – Task-Manifold Guidance

6.1 Paper Version (surface-to-surface)

The geometric objective is built on point clouds (Chamfer):

$$\mathcal{L} = d_{\text{Chamfer}}\big(T(\hat{a}_0)\,O^0,\ O^1\big). \tag{3}$$

The intermediate action is then corrected in the opposite gradient direction.

Implementation in `_apply_task_manifold_guidance`:

- Action → pose conversion: `action_to_se3`.
- Back-transform to local gripper frame, then apply $T(\hat{a}_0)$ (frame consistency).
- Chamfer gradient: `chamfer_distance_grad`.

6.2 Practical Version (point-to-point)

Guidance pushes toward an explicit `target_grasp_action` (target position + quaternion) using a simplified L2 error.

7 Step 5 – Spherical Constraint (Eq. 9)

The correction is normalized and scaled by the current noise level:

$$a_{t-1} = \tilde{a}_{t-1} - s\,\sqrt{d}\,\sigma_t\,\frac{\nabla_a \mathcal{L}}{\|\nabla_a \mathcal{L}\|}, \tag{4}$$

where d is the action dimension and s is a stabilization factor (in code: `spherical_scale`).

Implementation:

- `use_spherical=True`: enables the normalized update rule.
- `spherical_scale`: controls guidance step intensity.

8 Step 6 – Full Inference Loop

In `inference(...)`:

1. Choose initialization: task-aware (if `use_fgr`) or Gaussian.
2. Iterate decreasing t from `t_start` down to 0.
3. At each step: DDPM $\rightarrow$ clean estimate $\hat{a}_0$ $\rightarrow$ manifold guidance $\rightarrow$ spherical constraint.
4. Final output: action a_0 applied to the environment/controller.

9 Main Parameters and Their Role

- `implementation`: `paper` vs `practical`.
- `M`: initial noise level (smaller means closer-to-final initialization).
- `spherical_scale`: stability vs aggressiveness of guidance.
- `deterministic_denoising`: temporal stability in simulation and robot loops.
- `use_blend` (practical): additional stabilization toward explicit target.

10 Point Clouds in This Codebase

10.1 Simulation Pipeline

In simulation, point clouds are generated synthetically inside `env/panda_env.py` through `_get_obs()`:

- `pc_gripper` is built with `sample_gripper_pointcloud(T_ee, gripper_width,...)` from `utils/point_cloud.py`, using a geometric model of the Panda hand (palm + two fingers) transformed to world coordinates.
- `pc_scene` is built with `sample_scene_pointcloud(obj_pos,...)` from `utils/point_cloud.py`, as a concatenation of object points + table points.
- Both point clouds are random-sampled each step with the environment RNG, which provides variability while keeping reproducibility with a fixed seed.

10.2 How ADPro Uses Them

ADPro consumes these clouds from the observation dictionary (`obs['pc_gripper']`, `obs['pc_scene']`):

- `_task_aware_init`: subsamples both clouds (`subsample_pointcloud`) and estimates a structured initialization.
- `paper` mode: computes rigid alignment (FGR) and Chamfer-based geometric guidance from point-cloud correspondences.
- `practical` mode: uses cloud-derived object center + explicit grasp target for a more stable heuristic guidance.

10.3 Real-Robot Layer: Retrieved vs Reconstructed Clouds

In `real_robot/`, point-cloud handling depends on the backend:

- **ROS2 backend** (`adapters/ros2_franka_adapter.py`): `pc_gripper` and `pc_scene` are *retrieved* from ROS2 `PointCloud2` topics and converted with `_pc2_to_xyz`.
- **GELLO backend** (`adapters/gello_zmq_adapter.py`): robot joint states are retrieved from ZMQ, then `T_ee` is computed via FK, and point clouds are *reconstructed/simulated* with `sample_gripper_pointcloud` and `sample_scene_pointcloud`.
- **Mock backend** (`adapters/mock_adapter.py`): observations come directly from `PandaPickPlaceEnv`, so clouds follow the same synthetic generation path as simulation.

10.4 Practical Consequence

The policy interface remains identical (`pc_gripper`, `pc_scene`, `T_ee`, etc.), which allows ADPro inference to run unchanged across simulation and deployment backends.

11 Visualization and Evaluation in This Repository

11.1 Simulation and Offline Evaluation

In `simulate_realtime.py`, `-show-denoise` visualizes:

- internal denoising trajectory $a_t \to a_0$ (baseline vs ADPro),
- ADPro initialization state,
- guidance metrics (gradient norm, step norm, blend).

In `demo.py`, ADPro is benchmarked against baseline with:

- denoising convergence curves,
- action-component/backtracking plots,
- 3D executed trajectories,
- MSE vs denoising steps,
- success-rate vs denoising steps.

11.2 Real-Robot Layer (Updated)

The repository now includes a separated deployment layer under `real_robot/`:

- `control_loop.py`, `policy_engine.py`, `safety.py`: observation-policy-command loop with safety guards.
- Adapters: `mock_adapter.py` (local dry-run), `ros2_franka_adapter.py` (ROS2), `gello_zmq_adapter.py` (GELLO/ZMQ stack).
- Operational scripts: `run_policy.py`, `calibrate_workspace_guided.py`, `calibrate_local_frame_matrix.` and motion-test scripts.
- Config files: `real_robot/config/lab_franka.json` and `lab_franka.yaml`.

11.3 Calibration and Deployment Notes

Calibration is required to align camera/local coordinates with robot/world coordinates before deploying ADPro on hardware. The added scripts support workspace-bound estimation and local-frame matrix calibration to make execution reproducible and safe.

12 Summary

ADPro improves a diffusion policy by injecting geometric structure at test time: task-aware initialization, manifold guidance, and spherical-normalized updates. The `paper` variant maximizes theoretical fidelity, while the `practical` variant maximizes robustness in simulation.